

Encryption – Crypto 101 using TryHackMe Platform

Aim

To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Theory (Brief)

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.

Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type **Crypto 101**
2. Click on the room named **Crypto 101**
3. Click **Join Room**

Step 3: Start the Machine (if required)

1. Click **Start Machine / Start AttackBox**
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.

2. Open the TryHackMe terminal.
3. Give the command in the terminal prompt:
echo "Q3J5cHRvZ3JhcGh5" | base64 -d
4. Change the Input and get different outputs.
5. The output will be the decoded plain text.
6. Enter the decoded text into the answer box.

To Encode Base64

echo "cryptography" | base64

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

echo 48656c6c6f | xxd -r -p

Text to Hex

echo "Hello" | xxd -p

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

1. Identify the shift value (given or by trial).
2. Use online Caesar decoder or terminal tool.
3. Change the rotation type and get different answers.

Using terminal

echo "KHOOR" | tr 'A-Z' 'X-ZA-W'

(Shift = 3)

Expected Output: HELLO

(b) Shift-Based Encryption

Purpose:

Similar to Caesar cipher but may use different shift values.

How to Solve

1. Try different shift values until meaningful text appears.
2. Online tools can be used if allowed.

Example:

- Shift 1 → incorrect
- Shift 3 → readable text → correct answer

c) Playfair Cipher

Purpose: To encrypt using alphabets directly

How to Solve

1. In the terminal type nano playfaircipher.py
 Playfaircipher.py code
 # playfair_cipher.py
 # Usage:
 # python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"
 # python3 playfair_cipher.py dec -k "MONARCHY" -t

"BMNDZBXDKYBEJVDMUIXMMOUIVIF"

```
import argparse
import re
```

```
def normalize_text(s: str) -> str:
    s = re.sub(r"[^A-Za-z]", "", s).upper()
    s = s.replace("J", "I") # I/J combined
    return s
```

```
def build_square(key: str):
    key = normalize_text(key)
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ" # no J
    seen = set()
    seq = []
    for ch in key + alphabet:
        if ch not in seen:
            seen.add(ch)
            seq.append(ch)
    square = [seq[i:i+5] for i in range(0, 25, 5)]
    pos = {square[r][c]: (r, c) for r in range(5) for c in range(5)}
    return square, pos
```

```
def make_digraphs(plain: str, filler: str = "X"):
    plain = normalize_text(plain)
```

```

digs = []
i = 0
while i < len(plain):
    a = plain[i]
    b = plain[i+1] if i+1 < len(plain) else ""
    if b == "":
        digs.append((a, filler))
        i += 1
    elif a == b:
        digs.append((a, filler))
        i += 1
    else:
        digs.append((a, b))
        i += 2
return digs

```

```

def playfair_encrypt(plain: str, key: str) -> str:
    square, pos = build_square(key)
    out = []
    for a, b in make_digraphs(plain):
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb: # same row
            out.append(square[ra][(ca + 1) % 5])
            out.append(square[rb][(cb + 1) % 5])
        elif ca == cb: # same col
            out.append(square[(ra + 1) % 5][ca])
            out.append(square[(rb + 1) % 5][cb])
        else: # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

```

```

def playfair_decrypt(cipher: str, key: str) -> str:
    cipher = normalize_text(cipher)
    if len(cipher) % 2 != 0:
        raise ValueError("Ciphertext length must be even for Playfair.")
    square, pos = build_square(key)
    out = []
    for i in range(0, len(cipher), 2):
        a, b = cipher[i], cipher[i+1]
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb: # same row
            out.append(square[ra][(ca - 1) % 5])
            out.append(square[rb][(cb - 1) % 5])

```

```

        elif ca == cb: # same col
            out.append(square[(ra - 1) % 5][ca])
            out.append(square[(rb - 1) % 5][cb])
        else: # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("mode", choices=["enc", "dec"])
    ap.add_argument("-k", "--key", required=True)
    ap.add_argument("-t", "--text", required=True)
    args = ap.parse_args()

    if args.mode == "enc":
        print(playfair_encrypt(args.text, args.key))
    else:
        print(playfair_decrypt(args.text, args.key))

if __name__ == "__main__":
    main()

```

2. Save the file by giving Ctrl + X. It will ask whether to save. Give Y and save the text editor
3. Give the command to run playfair cipher
python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"
4. Give the following command to run decryption
python3 playfair_cipher.py dec -k "MONARCHY" -t "twqe kjl eazq"

d) Vigenère Cipher

Follow above steps and create vignerecipher.py. code in python and execute.

4.3 Hashing

(a) Understanding Hash Algorithms

Hash Type

Length

MD5	32 characters
-----	---------------

SHA-1	40 characters
-------	---------------

SHA-256	64 characters
---------	---------------

(b) Identifying Hashes

How to Solve

1. Count the number of characters in the given hash.
2. Match it with the table above.

Example:

5d41402abc4b2a76b9719d911017c592

→ 32 characters → MD5

Generating Hashes

echo -n "hello" | md5sum

echo -n "hello" | sha1sum

echo -n "hello" | sha256sum

4.4 Encryption Concepts

(a) Encryption vs Hashing

Encryption

Reversible

Uses keys

Used for secure communication

Hashing

Irreversible

No keys

Used for password storage

(b) Symmetric vs Asymmetric Encryption

Symmetric

One key

Fast

Example: AES

Asymmetric

Two keys

Slower

Example: RSA

Step 5: Verify Completion

1. Ensure all tasks are marked **Completed**
2. Observe the success message at the end of the room

Output

- Successfully decoded encrypted messages
- Paste the screen shots

Result

Thus, the **Crypto 101** experiment was successfully completed on the **TryHackMe platform**.

The experiment helped in understanding **basic cryptographic concepts**, encryption techniques, encoding methods, and hashing algorithms through hands-on practice.